

Formal Response to `verification.zip`: Review of the Computational Verification Package for the Repaired Split-Forest Proof

Prepared for Michael Wessel

May 2026

Abstract

I inspected and ran the supplied `verification.zip` package. The four included Python scripts are reproducible and all terminate with **PASS**. This is useful evidence: the RCC5 composition table is now strongly sanity-checked, several vertical-fragment regressions are covered, and the scripts exercise the intended separation between blocking and semantic equality in simple cases. However, the package is not yet a proof certificate for decidability. Its certificate checker covers only a vertical fragment, the multiple-superpart stress test is still too weak to force incomparable superpart witnesses, the request-cycle negative test is mostly a local type clash rather than a pure eventuality test, and the mosaic tests remain bounded triangle/hand-case checks rather than a proof of the required patchwork theorem. My conclusion is therefore: the verification package materially improves confidence, but the main theorem still requires additional formal or computational tightening before it should be treated as established.

1 Scope of this review

The reviewed archive contains the following files:

```
verification/README.md
verification/python/rcc5_composition_check.py
verification/python/certificate_checker.py
verification/python/small_model_oracle.py
verification/python/mosaic_closure_search.py
verification/reports/rcc5_composition_table.txt
verification/reports/certificate_checker.txt
verification/reports/small_model_oracle.txt
verification/reports/mosaic_closure_search.txt
```

I reproduced the advertised reports by running:

```
cd verification
python3 python/rcc5_composition_check.py
python3 python/certificate_checker.py
python3 python/small_model_oracle.py
python3 python/mosaic_closure_search.py
```

All four scripts exited successfully and reproduced the claimed **PASS** verdicts.

This response evaluates what those **PASS** results do and do not establish about the repaired split-forest proof. I focus on the proof obligations previously identified as fragile: the RCC5 composition

table, witness-generated thinning, generated cover trees, blocking versus semantic equality, multiple upward PP-witnesses, request-closed cycles, typed-EQ quotienting, and support-closed sibling mosaics.

2 Executive verdict

The verification package is a good regression suite and a useful sanity check. It does *not* close the proof.

The status should be described as follows:

Reproducible computational evidence: yes.

Publication-grade proof of decidability: not yet.

More specifically:

Component	Assessment
RCC5 composition table	Strongly sanity-checked by exhaustive finite-subset enumeration; this part is now much less concerning.
Vertical certificate checks	Useful, but only a hand-written corpus for a vertical fragment; not a full checker for the finite certificates in the proof.
Small-model oracle	Useful bounded counterexample search; cannot certify infinite regular models or completeness.
Mosaic tests	Helpful triangle and side-witness sanity checks; not a proof of the required patchwork/support-closure theorem.
Lean/formal proof targets	Not attempted. The most important proof obligations therefore remain unformalized.
Overall theorem	Promising route, but remaining gaps prevent treating decidability as fully established.

3 Positive findings

3.1 The RCC5 composition table is now well checked

The script `rcc5_composition_check.py` computes the RCC5 composition table by exhaustive enumeration of non-empty subsets of a finite universe. At universe size $|U| = 5$, all 25 cells match the table in the repaired proof. The script also reports stability of the computed table from $|U| = 4$ to $|U| = 5$.

This is strong computational evidence for the composition table. It also catches exactly the earlier kind of error: for example, it correctly gives

$$\text{DR} \circ \text{DR} = \{\text{EQ}, \text{DR}, \text{PO}, \text{PP}, \text{PPI}\},$$

not merely $\{\text{DR}\}$, and

$$\text{PP} \circ \text{PP} = \{\text{PP}\}.$$

I would regard this work package as essentially closed for manuscript purposes, except that a small finite proof or Lean formalization would still be useful.

3.2 The vertical-fragment checker exercises the right repair themes

The script `certificate_checker.py` checks several finite validity conditions for a vertical fragment: type legality, vertical safety, equality-aware vertical existential/universal discharge, a partial typed-equality condition, and exact summaries as represented in the script.

The test corpus does exercise three important repair themes:

- parent self-loops represent infinite unfoldings, not semantic EQ-cycles;
- explicit equality ports must not identify vertically comparable occurrences;
- multiple selected parents require equality-mate occurrences, not a single-parent profile.

This is useful evidence that the corrected architecture has been understood at the implementation level.

3.3 The finite oracle corroborates selected small examples

The script `small_model_oracle.py` enumerates complete finite abstract RCC5 labelings on domains up to size four, with reflexivity, converse, and triple composition. It confirms finite satisfiability for simple positive examples and no finite model for some negative examples. It also correctly treats

$$\exists PP.T \sqcap \forall PP.\exists PP.T$$

as an infinite-model-only positive case rather than as finite-model satisfiable.

This is good as a counterexample search oracle. It is not, by design, a completeness proof.

4 Remaining gaps and shortcomings

4.1 The package contains no Lean formalization

The README explicitly says that the Lean targets were not attempted. That is acceptable for a first verification pass, but it means the most important proof obligations remain hand proofs. In particular, the following are still not machine checked:

- the witness-generated submodel lemma;
- the typed-EQ quotient soundness lemma;
- the request-closed lasso/blocking lemma;
- the distinction between blocking repetition and semantic equality;
- the patchwork/support-closed mosaic theorem.

For increasing certainty, these should remain the next formal targets.

4.2 The certificate checker is a fixed vertical-fragment regression suite, not a full fuzzer

The checker itself states that conditions involving DR/PO mosaics are out of scope. In the script’s terminology, $V2, V6, V7, V8$ are not checked by `certificate_checker.py`. The corpus consists of ten hand-written certificates with hard-coded expected outcomes.

That is useful, but it is not a full validity checker for the finite certificates in the proof. In particular, it does not yet test the interaction among:

- vertical PP/PPI reachability;
- explicit equality mates;
- DR/PO side witnesses;
- support-closed mosaics;
- quotienting by semantic EQ.

A stronger tool would generate random and exhaustive small certificates, check all validity clauses, unfold them to finite depths, and search for violations of RCC5 composition, type propagation, equality congruence, or modal satisfaction.

4.3 The multiple-upward-PP-witness stress formula is still too weak

The current C_{AB} test in the verification package is essentially

$$A \sqcap B \sqcap \exists PP.A \sqcap \exists PP.B \sqcap \forall PP(\neg A \sqcup \neg B) \sqcap \forall PP(\neg B \sqcup \neg A).$$

This does not force the A -superpart and the B -superpart to be incomparable. It can be satisfied along a single PP-chain: the first superpart can satisfy $A \wedge \neg B$, and a higher superpart can satisfy $B \wedge \neg A$. Therefore the test does not fully exercise the hard case that motivated equality-mate split copies with different selected parents.

The intended stress formula should force any A -superpart and any B -superpart to be non-equal and non-comparable. A better NNF test is:

$$C_{AB}^{\text{inc}} := \exists PP.A \sqcap \exists PP.B \sqcap \forall PP \left((\neg A \sqcup (\neg B \sqcap \forall PP.\neg B \sqcap \forall PPI.\neg B)) \sqcap (\neg B \sqcup (\neg A \sqcap \forall PP.\neg A \sqcap \forall PPI.\neg A)) \right).$$

This formula allows an abstract model with two overlapping incomparable superparts, one carrying A and one carrying B , but it prevents a single comparable chain or a single $A \wedge B$ superpart from satisfying both existential requirements.

As an additional reviewer-side check, I evaluated this stronger concept with the supplied small-model oracle infrastructure. A size-three model is found: the root has two PP-successors related by PO, one carrying A and one carrying B . This stronger concept should be added to WP2 and WP3, together with a certificate that is valid only when equality-mate occurrences may have different parents.

4.4 The request-cycle negative test is not a pure eventuality test

The negative WP5 test in `certificate_checker.py` uses a concept of the form

$$\neg A \sqcap \exists PP.A \sqcap \forall PP.\neg A.$$

But $\forall PP.\neg A$ is the NNF complement of $\exists PP.A$. Thus the test is rejected at local type legality, before the request-discharge condition is genuinely exercised.

A better pure request-closure test is a one-state self-loop carrying

$$\neg A \sqcap \exists PP.A$$

with no state on the cycle satisfying A . This is locally type-consistent, but should fail the V4-style existential discharge condition. I checked this ad hoc against the supplied checker; it is rejected by V4, as desired. That test should be included explicitly in the verification corpus.

The corresponding positive test should be either:

- a two-state quotient cycle where some later cycle state carries A , or
- a stem state whose request is discharged before the cycle begins.

This would test request-closed blocking more directly than the current negative case.

4.5 The occurrence-level equality issue is only partially tested

The verification package correctly emphasizes that blocking repetition is not semantic equality. However, the script abstracts away occurrence indices. For example, strict reachability along a self-loop is represented by $s \text{ Reach}_{PP} s$, while the equality check intentionally skips some same-state cases because repeated laps unfold to fresh occurrences.

That abstraction is reasonable for a hand test, but it is exactly where the formal proof is delicate. The manuscript still needs a precise occurrence-level semantics proving that:

- each lap of a blocking cycle creates fresh occurrences;
- equality ports are instantiated per occurrence, not per profile name;
- profile equality never implies semantic EQ;
- explicit EQ-mates are quotiented only through the semantic equality relation.

The script gives evidence that the intended convention was implemented in simple examples. It does not prove that no leakage remains in the general certificate semantics.

4.6 The mosaic tests do not prove the patchwork theorem

The script `mosaic_closure_search.py` performs useful checks:

- enumeration of all 3-position off-diagonal relation triangles;
- finite set realizability of locally consistent triangles;
- a hand-written DR/PO side-witness insertion scenario;
- a simple PP-chain universal-propagation check;

- a sibling-branching example.

These are good sanity checks. They are not the patchwork theorem required by the proof. The proof needs something closer to:

For every finite prefix network generated by a valid finite split-forest certificate, if the relevant local mosaics are support-closed and agree on overlaps, then the full prefix network satisfies converse, typed equality, and RCC5 composition for all triples.

The current script checks triangles and selected hand cases. It does not implement arbitrary finite support-closed mosaic families with overlap maps, nor does it search for global synchronization failures among overlapping mosaics of arity greater than three. It also uses very partial candidate types for DR/PO side witnesses rather than full Hintikka types over the closure.

Therefore the correct status of WP6 is:

bounded sanity checks passed; patchwork theorem still required.

not:

mosaic proof obligation closed.

4.7 The small-model oracle is useful but bounded

The finite oracle searches domains up to size four. This is useful for discovering small counterexamples and corroborating finite satisfiable examples. However, it cannot establish completeness for regular infinite split forests. In particular, concepts such as

$$\exists PP.T \sqcap \forall PP.\exists PP.T$$

are intentionally outside finite-model search. The oracle's role should be described as bounded counterexample search, not as evidence for the infinite unfolding theorem except indirectly.

5 Recommended revisions to the verification package

I recommend the following concrete next steps.

5.1 Replace or supplement C_{AB} with an incomparability-forcing test

Add C_{AB}^{inc} from Section 4.3 to both `certificate_checker.py` and `small_model_oracle.py`. The certificate checker should include:

1. a valid certificate with two equality-mate root occurrences and two different selected parents, one satisfying A and one satisfying B ;
2. an invalid single-parent certificate;
3. preferably, an invalid certificate where the two witnesses are placed on one comparable chain.

5.2 Add a pure request-closure test

Add a one-state self-loop certificate for

$$\neg A \sqcap \exists PP.A$$

with no cycle state carrying A . It should fail existential discharge, not type legality. Also add a positive two-state or multi-state lasso where the request is actually fulfilled.

5.3 Integrate the mosaic checker with the certificate checker

The current certificate checker omits the mosaic clauses, while the mosaic checker tests standalone hand cases. A stronger checker should parse or construct a full finite certificate containing vertical structure, equality mates, DR/PO witness menus, and support-closed mosaics. It should then verify all certificate clauses in one place.

5.4 Add genuine random or exhaustive small-certificate fuzzing

The current corpus is hand-written. Add generators for small profile sets, parent graphs, equality-port assignments, types, and mosaic families. For each generated candidate:

- run the validity checker;
- unfold to bounded depth;
- check RCC5 composition, typed equality, vertical safety, modal satisfaction, and witness realization on the finite prefix.

Any valid certificate whose finite prefix violates a semantic condition is a concrete soundness bug.

5.5 State or formalize the patchwork theorem

The manuscript should either:

1. cite a precise RCC5 patchwork theorem and verify that the certificate's support-closed mosaics satisfy its hypotheses; or
2. give a self-contained proof that the support-closed mosaic construction globally synchronizes finite prefixes.

The current triangle enumeration is not a substitute for this theorem.

5.6 Begin Lean formalization with three core lemmas

The highest-value Lean targets remain:

1. witness-generated submodel preservation;
2. typed-EQ quotient soundness;
3. request-closed lasso/blocking soundness.

Formalizing these would increase confidence much more than further hand-written positive examples.

6 Revised status table

Obligation	Revised status after <code>verification.zip</code>
Correct RCC5 composition table	Strongly supported by reproducible enumeration.
Closure and NNF complement	Not directly tested here, but likely closed by manuscript proof and script usage.
Witness thinning	Still a hand proof; should be formalized.
Generated witness-cover interpretation	Conceptually resolved; computational package assumes it.
Blocking versus semantic equality	Good simple tests; needs occurrence-level formal proof.
Multiple upward PP witnesses	Not yet adequately tested; current formula is too weak.
Request-closed cycles	Partially tested; add pure eventuality test.
Typed-EQ quotient	Still a major hand proof obligation.
Support-closed mosaics	Sanity-checked on triangles/hand cases; patchwork theorem still open.
Finite certificate decidability	Plausible, but needs full grammar/checker and bounds.

7 Conclusion

The supplied verification package is a real improvement. It is reproducible, catches the earlier RCC5-table class of error, and provides useful regression tests for the repaired vertical-fragment architecture. It should be kept and expanded.

However, it should not be presented as closing the decidability proof. The most important remaining issues are:

1. the multiple-upward-PP-witness test must be strengthened to force incomparable superparts;
2. the request-closed lasso condition needs direct positive and negative tests that are not already local type clashes;
3. the DR/PO mosaic subsystem must be integrated with the full certificate checker;
4. the support-closed mosaic/patchwork theorem must be stated and proved, or precisely cited and instantiated;
5. the typed-EQ quotient and occurrence-level blocking semantics should be formalized, ideally in Lean.

My revised verdict is therefore:

The verification package increases confidence in the repaired proof architecture.
It does not yet establish decidability or close the proof obligations.